

Lab 4: Les variables

Durée: 15 minutes

Nous ne voulons pas hard-coder toutes nos valeurs dans le fichier `main.tf`. Nous pouvons créer un fichier de variables pour une utilisation plus facile.

- Tâche 1: créer des variables dans un bloc de configuration
- Tâche 2: interpoler ces variables
- Tâche 3: créer un fichier `terraform.tfvars`

Tâche 1: Créer un nouveau bloc de configuration pour les variables

Étape 1.1

Ajoutez trois variables en haut de votre fichier de configuration:

```
variable "access_key" {}  
variable "secret_key" {}  
variable "region" {  
    default = "<REGION>"  
}
```

Tâche 2: Interpoler ces variables dans votre code existant

Étape 2.1

Mettez à jour le bloc `provider` pour remplacer les valeurs hard-coded par les nouvelles variables.

```
provider "aws" {  
    access_key = var.access_key  
    secret_key = var.secret_key  
    region     = var.region  
}
```

Étape 2.2

Réexécutez `terraform plan` pour que Terraform récupère les nouvelles variables. Notez que vous serez invité à les entrer maintenant.

```
terraform plan  
  
var.access_key  
  Enter a value:
```

Utilisez Ctrl+C pour quitter.

Tâche 3: Modifiez votre fichier terraform.tfvars

Les variables définies uniquement dans le fichier de configuration seront invitées à la saisie. nous pouvons éviter cela en créant un fichier de variables.

Étape 3.1

Créez le fichier `terraform.tfvars` contenant ces deux premières paires clé-valeur:

```
access_key = "<ACCESSKEY>"
secret_key = "<SECRETKEY>"
```

Réexécutez `terraform plan` et notez que cela ne vous invite plus à saisir.

Étape 3.2

Ajoutez les variables correspondantes à votre fichier `main.tf` dans `terraform.tfvars`:

```
variable "access_key" {}
variable "secret_key" {}
variable "region" {
  default = "<REGION>"
}
variable "ami" {}
variable "subnet_id" {}
variable "identity" {}
variable "vpc_security_group_ids" {
  type = list
}
```

Étape 3.3

Modifiez également votre bloc de ressources avec ces modifications.

```
resource "aws_instance" "web" {
  ami              = var.ami
  instance_type    = "t2.micro"
  subnet_id        = var.subnet_id
  vpc_security_group_ids = var.vpc_security_group_ids

  tags = {
    "Identity"    = var.identity
    "Name"         = "Student"
    "Environment" = "Training"
  }
}
```

Après avoir effectué ces modifications, réexécutez `terraform plan`. Vous devriez voir qu'il n'y a aucun changement à appliquer, ce qui est correct, car les variables contiennent les mêmes valeurs que nous avons précédemment codées:

```
...

No changes. Infrastructure is up-to-date.

This means that Terraform did not detect any differences between your
configuration and real physical resources that exist. As a result, no
actions need to be performed.
```

Exercice

Nous allons démarrer à partir du fichier `instance.tf` suivant:

```
provider "aws" {
  access_key = "ACCESS_KEY_HERE"
  secret_key = "SECRET_KEY_HERE"
  region     = "us-east-1"
}

resource "aws_instance" "example" {
  ami           = "ami-0d729a60"
  instance_type = "t2.micro"
}
```

1. Divisez-le en quatre fichiers:

- **provider.tf**: il ne contiendra que le provider, qui est AWS, les secrets et la région. Et nous n'allons pas les remplir, nous allons nous référer à une variable.

```
provider "aws" {
  access_key = var.AWS_ACCESS_KEY
  secret_key = var.AWS_SECRET_KEY
  region     = var.AWS_REGION
}
```

- **vars.tf**: dans "vars.tf" nous allons déclarer les variables.

```
variable "AWS_ACCESS_KEY" {}
variable "AWS_SECRET_KEY" {}
variable "AWS_REGION" {
  default = "eu-west-1"
}
```

Si les accolades sont vides, nous ne définissons pas de valeur. Si nous spécifions un "default" est égal à quelque chose, nous définissons une valeur par défaut.

- **terraform.tfvars** : stocker les valeurs des variables.

```
AWS_ACCESS_KEY="<YOU_ACCESS_KEY>"
AWS_SECRET_KEY="<YOUR_SECRET_KEY>"
AWS_REGION="<YOUR_REGION>"
```

- **instance.tf** : contient la description de la ressource à créer sur AWS

```
resource "aws_instance" "example" {
  ami = ""
  instance_type = "t2.micro"
}
```

Nous allons créer des instances EC2 à partir de l'image Ubuntu Xenial, vous pouvez vous référer à [Amazon EC2 AMI Locator](#) pour retrouver les images selon les regions.

2. Essayer le déploiement de l'infrastructure
3. On voudrai avoir la possibilité de déployer cette infrastructure dans différentes régions, sachant que l'image de machine ami peut avoir différent ID selon la région. Pour cela:
 - a. Dans **vars.tf** déclarer une variable **AMIS** de type **map** de string dans laquelle vous associer à chaque region l'id de l'image disponible pour Ubuntu Xenial

```
variable "AMIS" {
  type = map(string)
  default = {
    us-east-1 = "<AWS_AMI_ID>"
    us-west-2 = "<AWS_AMI_ID>"
    eu-west-1 = "<AWS_AMI_ID>"
  }
}
```

- b. modifier l'**ami** dans **instance.tf** pour retrouver sa valeur selon la region à partir de la variable **AMIS**, avec la fonction **lookup**:

```
ami = lookup(var.AMIS, var.AWS_REGION, "") #last parameter is the default value
```